

Отчёт по лабораторной работе №8

Архитектура компьютеров и операционные системы

Степан Андреевич Гусев

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
5	Выводы	15
6	Ответы на контрольные вопросы	16
7	Список литературы	18

Список иллюстраций

4.1	Вход в систему	9
4.2	Запись в файл	9
4.3	Дописывание в файл	10
4.4	Запись в файл	10
4.5	Поиск файлов	10
4.6	Вывод файлов (команда)	10
4.7	Вывод файлов	11
4.8	Запуск фонового процесса	11
4.9	Удаление файла	11
4.10	Запуск редактора в фоновом режиме	12
4.11	Определение идентификатора процесса ps	12
4.12	Определение идентификатора процесса pgrep	12
4.13	man kill	12
4.14	Добавление права для файла	13
4.15	Получение информации	13
4.16	df -h	13
4.17	du -sh ~	13
4.18	man find	14
4.19	Вывод имён директорий	14

Список таблиц

1 Цель работы

Ознакомление с инструментами поиска файлов и фильтрации текстовых данных. Приобретение практических навыков: по управлению процессами (и заданиями), по проверке использования диска и обслуживанию файловых систем.

2 Задание

- 1) Осуществите вход в систему, используя соответствующее имя пользователя.
- 2) Запишите в файл `file.txt` названия файлов, содержащихся в каталоге `/etc`. Допишите в этот же файл названия файлов, содержащихся в вашем домашнем каталоге.
- 3) Выведите имена всех файлов из `file.txt`, имеющих расширение `.conf`, после чего запишите их в новый текстовый файл `conf.txt`.
- 4) Определите, какие файлы в вашем домашнем каталоге имеют имена, начинавшиеся с символа `c`? Предложите несколько вариантов, как это сделать.
- 5) Выведите на экран (постранично) имена файлов из каталога `/etc`, начинающиеся с символа `h`.
- 6) Запустите в фоновом режиме процесс, который будет записывать в файл `~/logfile` файлы, имена которых начинаются с `log`.
- 7) Удалите файл `~/logfile`.
- 8) Запустите из консоли в фоновом режиме редактор `gedit`.
- 9) Определите идентификатор процесса `gedit`, используя команду `ps`, конвейер и фильтр `grep`. Как ещё можно определить идентификатор процесса?
- 10) Прочтите справку (`man`) команды `kill`, после чего используйте её для завершения процесса `gedit`.
- 11) Выполните команды `df` и `du`, предварительно получив более подробную

информацию об этих командах, с помощью команды `man`.

- 12) Воспользовавшись справкой команды `find`, выведите имена всех директорий, имеющих в вашем домашнем каталоге.

3 Теоретическое введение

В интерфейсе командной строки есть очень полезная возможность перенаправления (переадресации) ввода и вывода (англ. термин I/O Redirection). Как мы уже заметили, многие программы выводят данные на экран. А ввод данных в терминале осуществляется с клавиатуры. С помощью специальных обозначений можно перенаправить вывод многих команд в файлы или иные устройства вывода (например, распечатать на принтере). Тоже самое и со вводом информации, вместо ввода данных с клавиатуры, для многих программ можно задать считывание символов их файла. Кроме того, можно даже вывод одной программы передать на ввод другой программе.

К каждой программе, запускаемой в командной строке, по умолчанию подключено три потока данных:

STDIN (0) — стандартный поток ввода (данные, загружаемые в программу). STDOUT (1) — стандартный поток вывода (данные, которые выводит программа). По умолчанию — терминал. STDERR (2) — стандартный поток вывода диагностических и отладочных сообщений (например, сообщениях об ошибках). По умолчанию — терминал.

Pipe (конвейер) — это однонаправленный канал межпроцессного взаимодействия. Термин был придуман Дугласом Макилроем для командной оболочки Unix и назван по аналогии с трубопроводом. Конвейеры чаще всего используются в shell-скриптах для связи нескольких команд путем перенаправления вывода одной команды (stdout) на вход (stdin) последующей, используя символ конвейера „|“.

4 Выполнение лабораторной работы

Вошёл в систему (рис. 4.1).

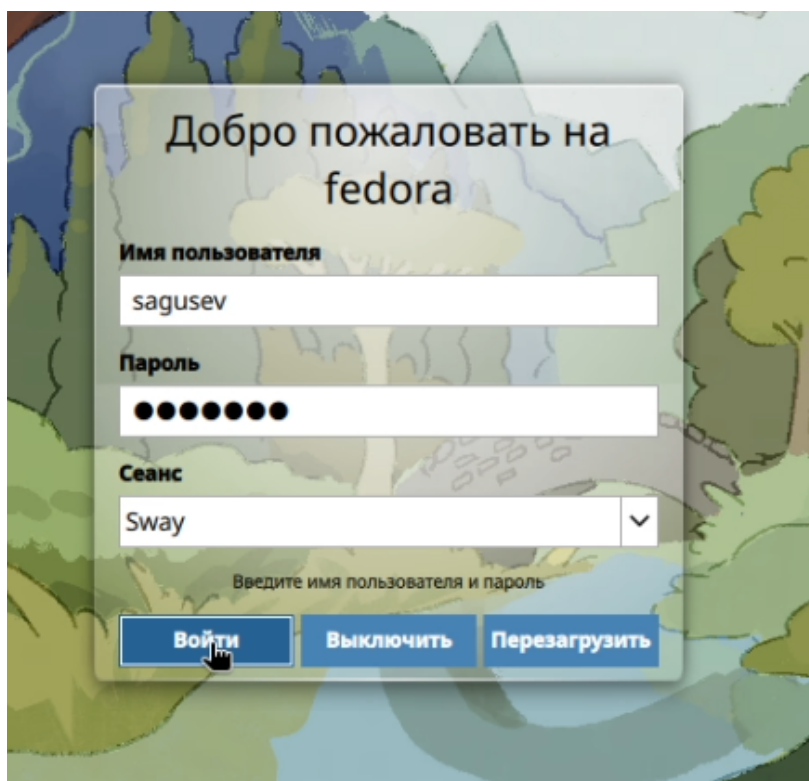


Рисунок 4.1: Вход в систему

Записал в файл file.txt названия файлов, содержащихся в каталоге /etc (рис. 4.2).

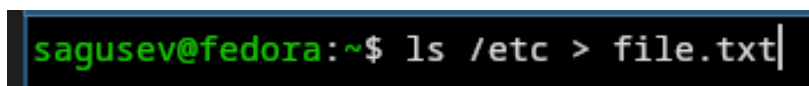


Рисунок 4.2: Запись в файл

Дописал в этот же файл названия файлов, содержащихся в домашнем каталоге (рис. 4.3).

```
sagusev@fedora:~$ ls >> file.txt
```

Рисунок 4.3: Дописывание в файл

Вывел имена всех файлов из file.txt, имеющих расширение .conf, после чего записал их в новый текстовый файл conf.txt (рис. 4.4).

```
sagusev@fedora:~$ grep "\.conf" file.txt > conf.txt
```

Рисунок 4.4: Запись в файл

Определил, какие файлы в домашнем каталоге имеют имена, начинавшиеся с символа „с“ двумя способами (рис. 4.5).

```
sagusev@fedora:~$ ls | grep "^с"
сconf.txt
sagusev@fedora:~$ find ~ -maxdepth 1 -type f -name "с*"
/home/sagusev/сconf.txt
```

Рисунок 4.5: Поиск файлов

Вывел на экран постранично имена файлов из каталога /etc, начинающиеся с символа „h“ (рис. 4.6), (рис. 4.7).

```
sagusev@fedora:~$ ls /etc/h* | less
```

Рисунок 4.6: Вывод файлов (команда)

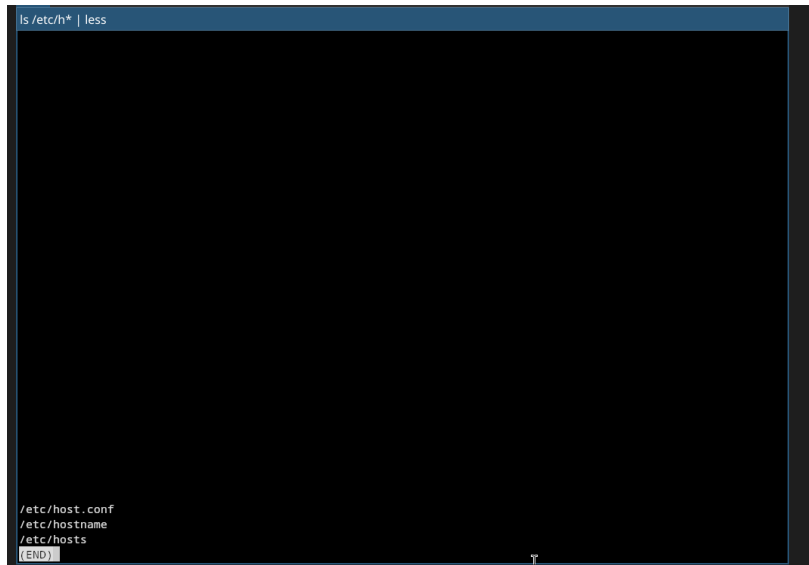


Рисунок 4.7: Вывод файлов

Запустил в фоновом режиме процесс, который записывает в файл ~/logfile файлы, имена которых начинаются с „log“ (рис. 4.8).

```
sagusev@fedora:~$ find / -name 'log*' -type f > ~/logfile 2>/dev/null &  
[1] 3702
```

Рисунок 4.8: Запуск фонового процесса

Удалил файл ~/logfile. (рис. 4.9).

```
sagusev@fedora:~$ rm logfile
```

Рисунок 4.9: Удаление файла

Запустил из консоли в фоновом режиме редактор nano вместо gedit, т.к. он не установлен (рис. 4.10).

```
sagusev@fedora:~$ gedit &
[1] 3953
bash: gedit: команда не найдена
[1]+ Выход 127      gedit
sagusev@fedora:~$ nano &
[1] 3980
```

Рисунок 4.10: Запуск редактора в фоновом режиме

Определил идентификатор процесса nano, используя команду ps, конвейер и фильтр grep (рис. 4.11).

```
sagusev@fedora:~$ ps aux | grep nano | grep -v grep
sagusev    3980  0.0  0.0 232564  3960 pts/0    T   11:40   0:00 nano
[1]+ Остановлен    nano
```

Рисунок 4.11: Определение идентификатора процесса ps

Ещё можно определить идентификатор процесса с помощью команды pgrep (рис. 4.12).

```
sagusev@fedora:~$ pgrep nano
3980
```

Рисунок 4.12: Определение идентификатора процесса pgrep

Прочёл справку команды kill (рис. 4.13).

```
sagusev@fedora:~$ man kill
```

Рисунок 4.13: man kill

Использовал kill для завершения процесса nano (рис. 4.14).

```
sagusev@fedora:~$ kill 3980
sagusev@fedora:~$ pgrep nano
3980
sagusev@fedora:~$ kill -9 3980
[1]+  Убито                  nano
sagusev@fedora:~$ pgrep nano
```

Рисунок 4.14: Добавление права для файла

Получил информацию о df и du, с помощью команды man (рис. 4.15).

```
sagusev@fedora:~$ man df
sagusev@fedora:~$ man du
```

Рисунок 4.15: Получение информации

Выполнил команду df -h, определив объем свободной памяти на жёстком диске (рис. 4.16).

```
sagusev@fedora:~$ df -h
Файловая система  Размер  Использовано  Дост  Использовано%  Смонтировано в
/dev/sda3          79G      9,2G        69G      12% /
devtmpfs           2,0G      0          2,0G       0% /dev
tmpfs              2,0G      368K        2,0G       1% /dev/shm
efivarfs           256K      178K        74K       71% /sys/firmware/efi/efivars
tmpfs              796M      1,2M       795M       1% /run
tmpfs              1,0M      0          1,0M       0% /run/credentials/systemd-journald.service
tmpfs              2,0G      4,0K        2,0G       1% /tmp
/dev/sda3          79G      9,2G        69G      12% /home
/dev/sda2          974M     314M       594M      35% /boot
/dev/sda1          599M      28M       580M       4% /boot/efi
tmpfs              1,0M      0          1,0M       0% /run/credentials/systemd-resolved.service
work               1,9T     319G      1,6T      17% /media/sf_work
tmpfs              398M      84K       398M       1% /run/user/1000
```

Рисунок 4.16: df -h

Выполнил команду du -sh ~, определив объем домашнего каталога (рис. 4.17).

```
sagusev@fedora:~$ du -sh ~
1,2G    /home/sagusev
```

Рисунок 4.17: du -sh ~

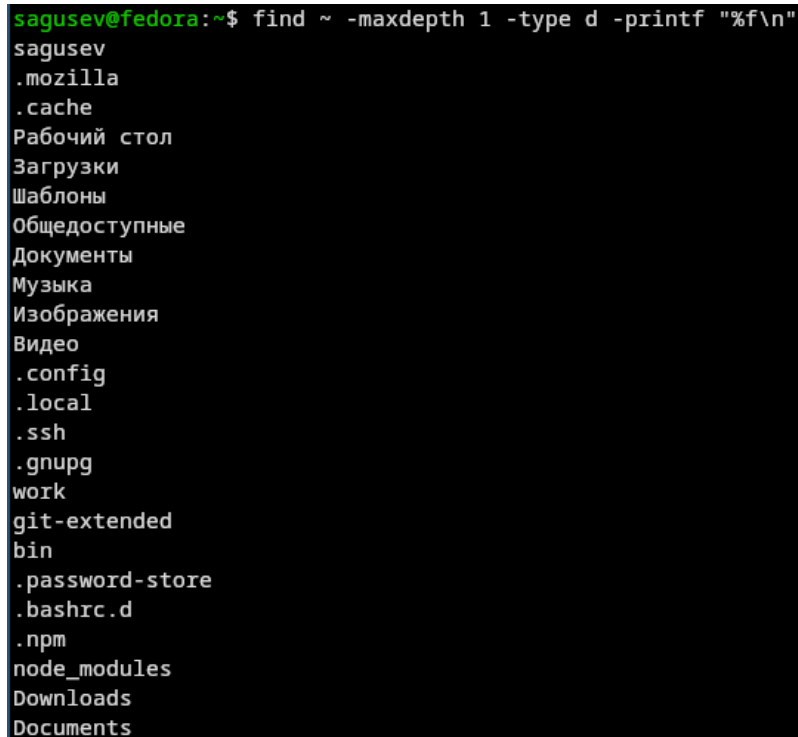
Прочёл справку команды find (рис. 4.18).



```
sagusev@fedora:~$ man find
```

Рисунок 4.18: man find

Вывел имена всех директорий, имеющихся в домашнем каталоге с помощью find (рис. 4.19).



```
sagusev@fedora:~$ find ~ -maxdepth 1 -type d -printf "%f\n"
sagusev
.mozilla
.cache
Рабочий стол
Загрузки
Шаблоны
Общедоступные
Документы
Музыка
Изображения
Видео
.config
.local
.ssh
.gnupg
work
git-extended
bin
.password-store
.bashrc.d
.npm
node_modules
Downloads
Documents
```

Рисунок 4.19: Вывод имён директорий

5 Выводы

Я ознакомился с инструментами поиска файлов и фильтрации текстовых данных. Я приобрёл практические навыки по управлению процессами (и заданиями), по проверке использования диска и обслуживанию файловых систем.

6 Ответы на контрольные вопросы

- 1) В системе по умолчанию открыто три специальных потока: – stdin – стандартный поток ввода (по умолчанию: клавиатура), файловый дескриптор 0; – stdout – стандартный поток вывода (по умолчанию: консоль), файловый дескриптор 1; – stderr – стандартный поток вывод сообщений об ошибках (по умолчанию: консоль), файловый дескриптор 2.
- 2) Этот знак > - перенаправление ввода/вывода, а » - перенаправление в режиме добавления.
- 3) Конвейер (pipe) служит для объединения простых команд или утилит в цепочки, в которых результат работы предыдущей команды передаётся последующей.
- 4) Чем это понятие отличается от программы? Главное отличие между программой и процессом заключается в том, что программа - это набор инструкций, который позволяет ЦПУ выполнять определенную задачу, в то время как процесс - это исполняемая программа.
- 5) PPID - (parent process ID) идентификатор родительского процесса. Процесс может порождать и другие процессы. UID, GID - реальные идентификаторы пользователя и его группы, запустившего данный процесс.
- 6) Запущенные фоном программы называются задачами (jobs). Ими можно управлять с помощью команды jobs, которая выводит список запущенных в данный момент задач.

- 7) Команда `htop` похожа на команду `top` по выполняемой функции: они обе показывают информацию о процессах в реальном времени, выводят данные о потреблении системных ресурсов и позволяют искать, останавливать и управлять процессами.

У обеих команд есть свои преимущества. Например, в программе `htop` реализован очень удобный поиск по процессам, а также их фильтрация. В команде `top` это не так удобно — нужно знать кнопку для вывода функции поиска.

Зато в `top` можно разделять область окна и выводить информацию о процессах в соответствии с разными настройками. В целом `top` намного более гибкая в настройке отображения процессов.

- 8) Команда `find` - это одна из наиболее важных и часто используемых утилит системы Linux. Это команда для поиска файлов и каталогов на основе специальных условий. Ее можно использовать в различных обстоятельствах, например, для поиска файлов по разрешениям, владельцам, группам, типу, размеру и другим подобным критериям.

Утилита `find` предустановлена по умолчанию во всех Linux дистрибутивах, поэтому вам не нужно будет устанавливать никаких дополнительных пакетов. Это очень важная находка для тех, кто хочет использовать командную строку наиболее эффективно.

Команда `find` имеет такой синтаксис: `find [папка] [параметры] критерий шаблон [действие]` Пример: `find /etc -name "p*" -print`

- 9) `find / -type f -exec grep -H „текстДляПоиска“ {} ;`
- 10) С помощью команды `df -h`.
- 11) С помощью команды `du -s`.
- 12) С помощью команды `kill%` номер задачи.

7 Список литературы

1. https://esystem.rudn.ru/pluginfile.php/3097173/mod_resource/content/4/006-lab_proc.pdf